System Architecture

The purpose of this document is to outline the architectural design and system components of the proposed **IoT-Based Smart Home System**. This system aims to provide a secure, efficient, and user-friendly home monitoring environment by integrating environmental sensing, automated resource management, and AI-supported security features.

The design prioritizes an **"Edge-First"** approach. Critical triggers and initial processing are performed locally on the device to ensure reliability and privacy, while cloud services are leveraged for heavy processing (AI Face Matching), data persistence, and remote accessibility.

The high-level system architecture consists of the following key nodes:

- **Edge Layer (Raspberry Pi 5):**The Raspberry Pi 5 operates as the local edge processing unit, running the Sensor Driver Service and Rule Engine. It communicates with the cloud over Wi-Fi using secure protocols (HTTPS/MQTT). The system is designed to support *graceful degradation*, ensuring that critical local alarms remain functional even when internet connectivity is unavailable.
- **Cloud Backend:** The cloud backend hosts the REST API and database services. It is responsible for centralized state management, data persistence, and triggering push notifications via an external notification provider when critical events occur.
- **Client Applications:** The Web Dashboard and Mobile Application fetch data from the cloud node via API requests, allowing users to view real-time status and historical logs.

1. Design of Home Controller

The hardware architecture is centered around a **Edge Layer (Raspberry Pi 5)** single-board computer, acting as the main home controller and edge processing unit. The system integrates various sensors and actuators through specific digital and analog interfaces to minimize latency.

- **Main Controller (Raspberry Pi 5):** Selected for its high processing power, enabling local sensor management and preliminary event detection.
- **Vision Module:** A Raspberry Pi Camera Module connected via the **CSI (Camera Serial Interface)** port to provide high-bandwidth video data for the surveillance system.
- **Environmental Sensors:** The **DHT11** (Temperature & Humidity) and **MQ-2** (Gas & Smoke) sensors are connected via GPIO pins to facilitate climate tracking and emergency detection.
- **Resource Monitoring:** A Load Cell paired with an **HX711 Amplifier Module** connects to digital pins to precisely measure weight for the "Automated Pet Resource Monitoring" feature.
- **Storage:** A high-speed microSD Card (64GB) is utilized for the OS, local logs, and image buffering during offline operation (Graceful Degradation).

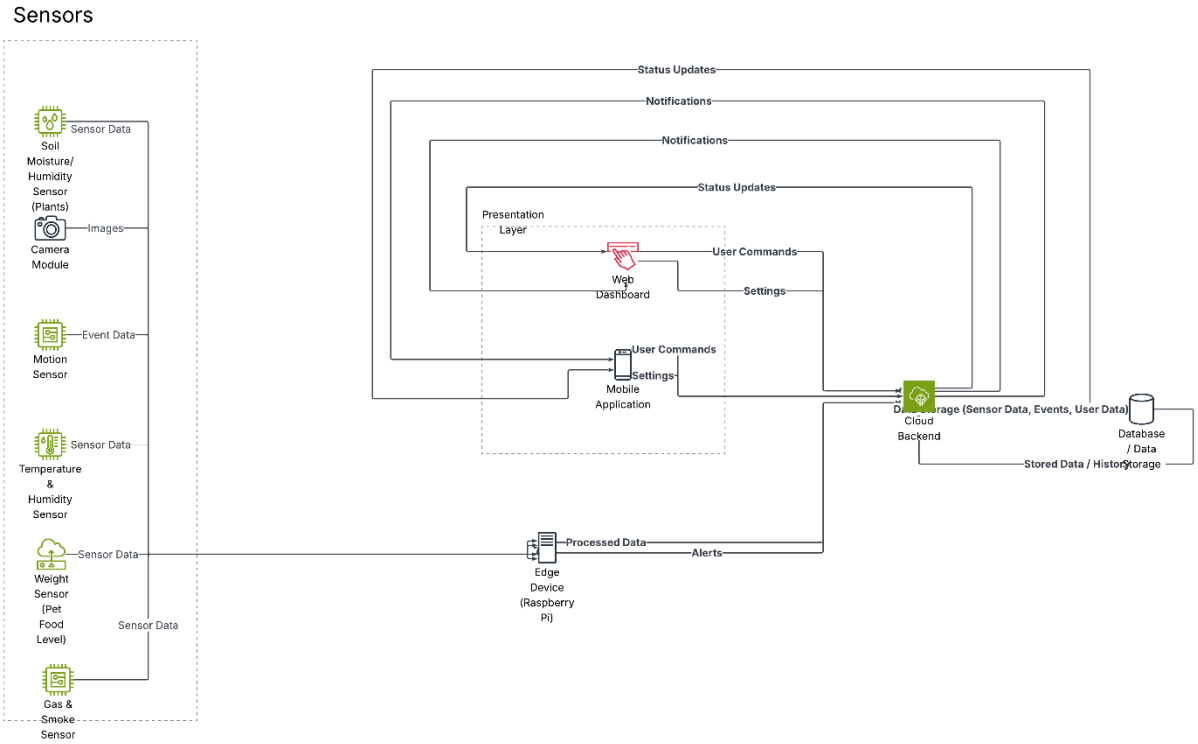

Figure 1 Design of Home Controller

2. Software Architecture Design

The software architecture follows a modular **4-Tier Layered Architecture** pattern to ensure separation of concerns, scalability, and maintainability.

2.1 Layer Decomposition

Layer 1: Presentation (User Interfaces)

This layer handles user interactions via the Mobile Application and Web Dashboard. It communicates solely through API calls, ensuring a stateless and responsive user experience. It allows users to view dashboards, receive notifications, and send manual commands (e.g., "Open Camera").

Figure 2 Mobile Application Security Notification

This figure illustrates a real-time security notification generated by the mobile application of the smart home system. When motion is detected and the captured individual cannot be identified as an authorized user, the system triggers an alert labeled *"Unknown Person Detected."* The notification provides a brief description of the event, enabling the user to take immediate action through the application.

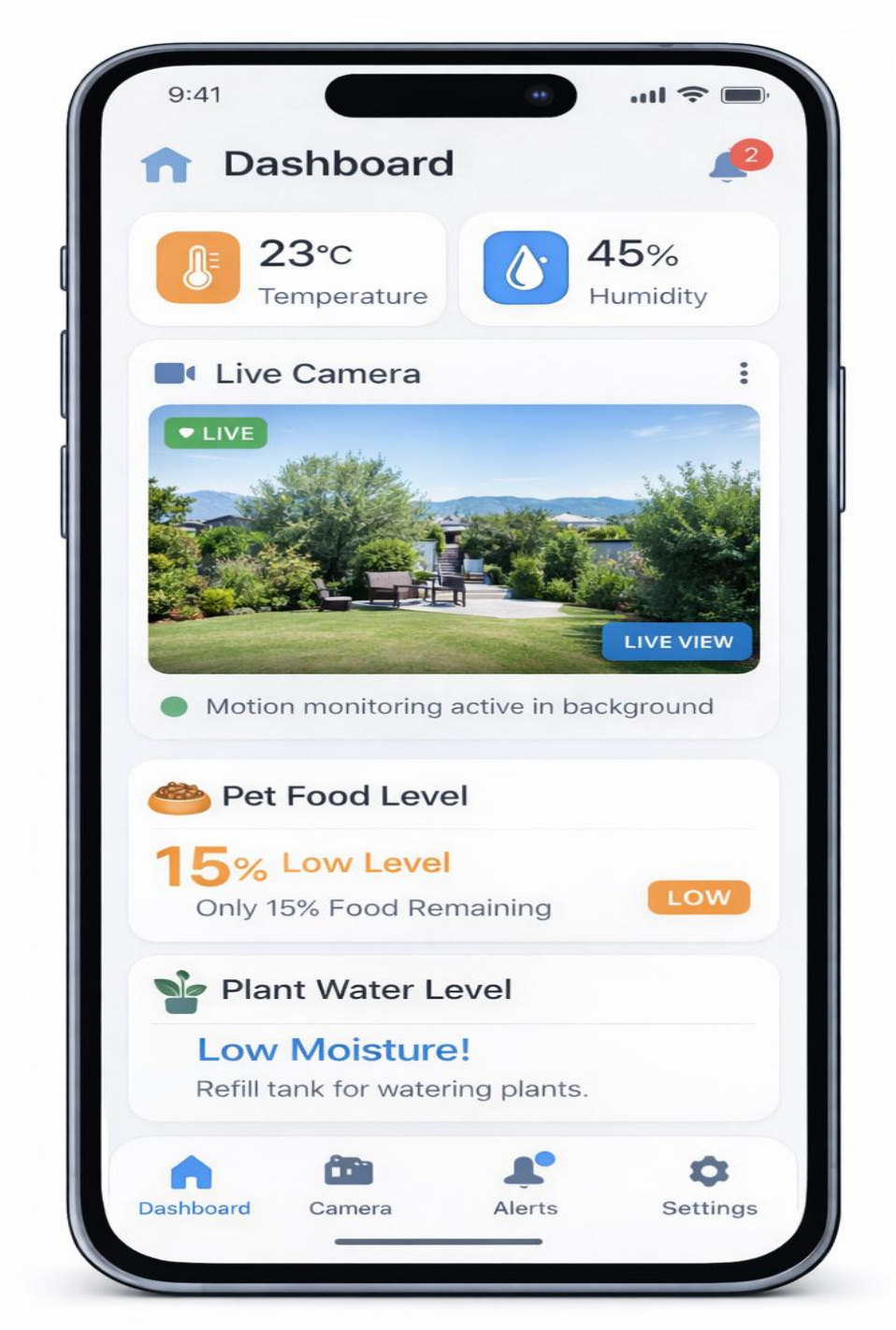

Figure 3 Mobile Application Dashboard Interface

This figure presents the main dashboard of the mobile application used in the smart home system. The interface displays real-time environmental data, including temperature and humidity levels, along with a live camera feed for continuous surveillance. Additionally, the dashboard provides status information for automated resource monitoring features such as pet food level and plant water level. Visual indicators and alerts are used to notify the user when critical thresholds are reached, enabling timely user intervention and efficient home management.

Figure 4 Web Dashboard Interface of the Smart Home System

This figure illustrates the web-based dashboard of the IoT smart home system. The interface provides a centralized view of real-time environmental data, including temperature and humidity levels, along with a live camera stream for continuous monitoring. In addition, the dashboard displays recent system alerts such as fire detection, intrusion events, and low resource warnings. Interactive controls and visual indicators enable users to acknowledge alerts, monitor system status, and manage home resources efficiently through a single web interface.

Layer 2: Security

This layer is responsible for user authentication during application login. Users are required to provide valid credentials (e.g., username and password) before accessing the system. Once authenticated, secure communication between the client applications and the backend is established using HTTPS/TLS. The security layer ensures that only authorized users can access system functionalities, while all business logic operations are executed after successful login verification.

Layer 3: Business Logic (Core Modules)

This is the core of the system where data processing occurs. To maintain a clean architecture and clear separation of duties, the functional requirements are grouped into four distinct handler modules:

1. **Climate Monitoring Service:** Handles periodic reading and logging of temperature and humidity data.
2. **Pet Resource Manager:** Manages the monitoring of food/water bowls and triggers alerts for low levels.
3. **Emergency Response Manager:** Dedicated to interrupt-based critical tasks like "Detecting Fire and Flood" to ensure immediate local feedback.
4. **Surveillance & Intrusion Analysis Engine:** A comprehensive security engine that handles the single-loop logic for motion capture, cloud-based face recognition, and the "Frequency Analysis" of unknown visitors.

Layer 4: Data & Integration

This layer manages persistence via the **Cloud Database** (storing user profiles, logs, and safe/non-safe face vectors) and handles third-party integrations, such as the External Push Notification Provider.

3. Detailed Module Design & Sequence Flows

This section details the internal algorithms, logic flows, and interaction sequences for the key functions defined in the Business Logic Layer.

3.1 Climate Monitoring Service

Addresses Requirements: `[Req. 2.1.1]`

Since continuous data streaming places unnecessary load on the network, this module utilizes a **"Polling Mechanism"**.

- **Logic:** The system wakes up at a configurable interval (default: 30s), reads the DHT11 sensor via GPIO, validates the data range, and transmits the packaged JSON payload to the Cloud Backend.
- **Sequence:** The interaction flow for periodic climate tracking is illustrated below.

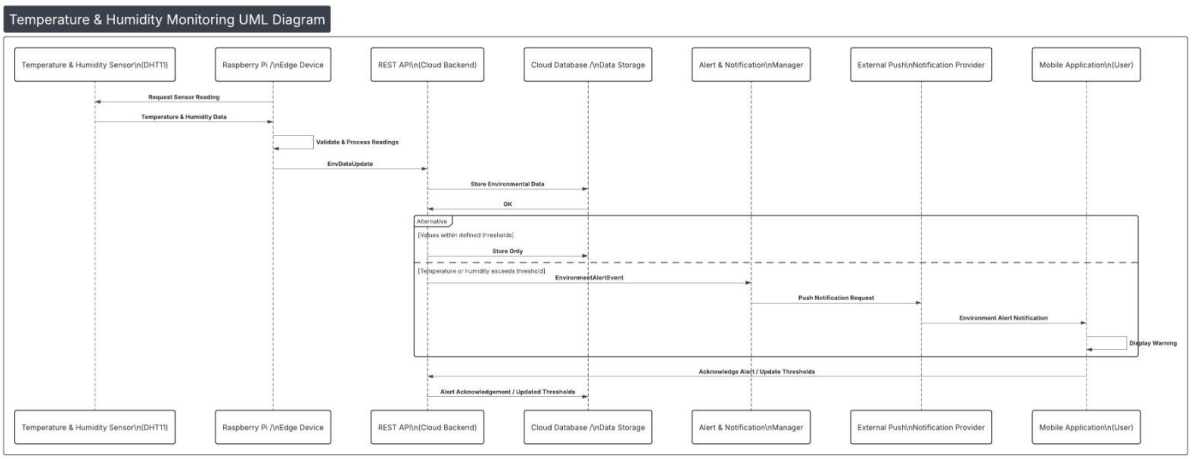

Figure 5 Temperature & Humidity Sequence Diagram

3.2 Pet Resource Manager

Addresses Requirements: `[Req. 2.1.4]`

This module focuses on ensuring pet safety by tracking resource availability.

- **Logic:** The system reads the Load Cell (via HX711). The analog weight is converted to a percentage (0-100%). If `Weight < Threshold (10%)`, a **"Low Resource Event"** is flagged. It includes a cooldown timer to prevent notification fatigue (e.g., only one alert every 4 hours).
- **Sequence:** The flow for weight measurement and alert triggering is shown below.

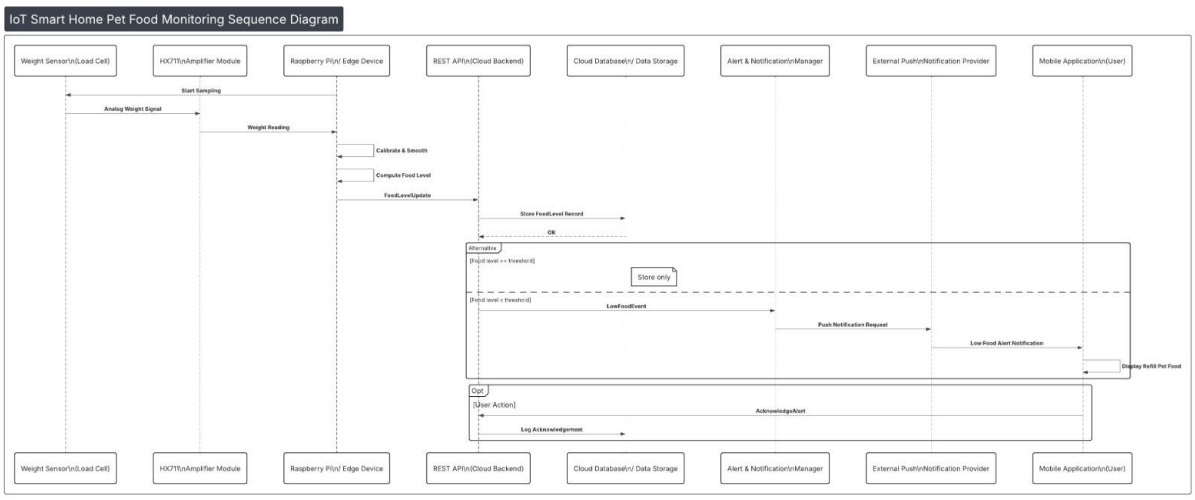

Figure 6 Pet Food Monitoring Sequence Diagram

3.3 Emergency Response Manager

Addresses Requirements: `[Req. 2.1.2]` & `[Req. 3.2]`

Unlike the polling mechanism, emergency sensors operate on an **Interrupt-based Architecture** to ensure immediate response.

- **Logic:** When the Gas/Smoke sensor triggers, the main loop is interrupted. The Edge Node immediately triggers the local buzzer (Graceful Degradation) while simultaneously sending a high-priority alert to the Cloud Backend for push notifications.
- **Sequence:** The flow from detection to user notification is shown below.

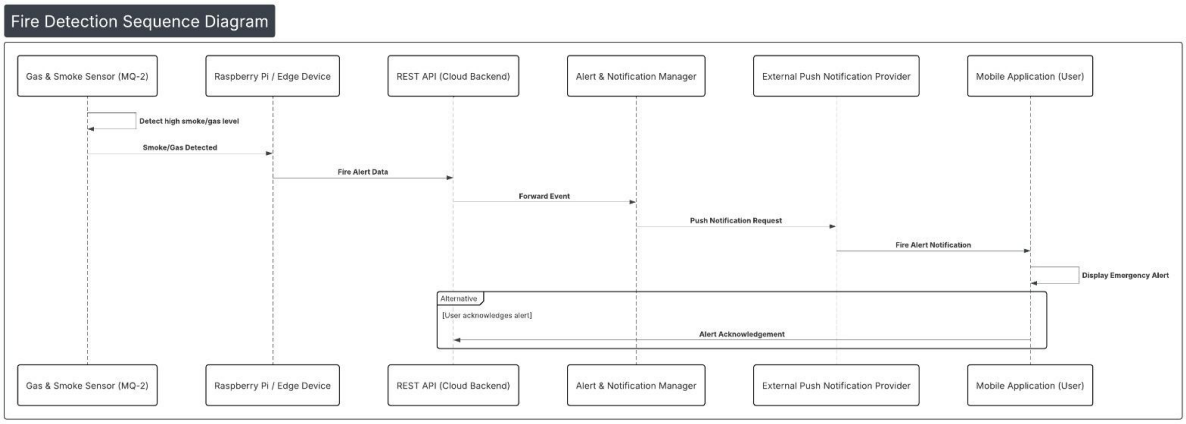

Figure 7 Fire Detection Sequence Diagram

3.4 Surveillance & Intrusion Analysis Engine

Addresses Requirements: `[Req. 2.2.1]`, `[Req. 2.2.2]`, `[Req. 2.4.3]`

The security system operates on a **Continuous Single-Loop Mechanism** to avoid complex multi-threading issues and ensure stability.

- **Logic Flow:**

1. **Sequential Check:** The system first checks for a "Manual Stream Request" from the app. If none, it checks the PIR Motion Sensor.
2. **Capture & Filter:** If motion is detected, images are captured. A local lightweight model checks for human presence.
3. **Cloud Identification:** Images are sent to the cloud. The AI compares the face against the "Safe User" database.
4. **Frequency Analysis:** If unauthorized, the system checks visit history (Frequency Analysis) and notifies the user with a visit count, asking for verification.

- **Sequence:** The complete flow from motion detection to AI classification and user notification is detailed below.

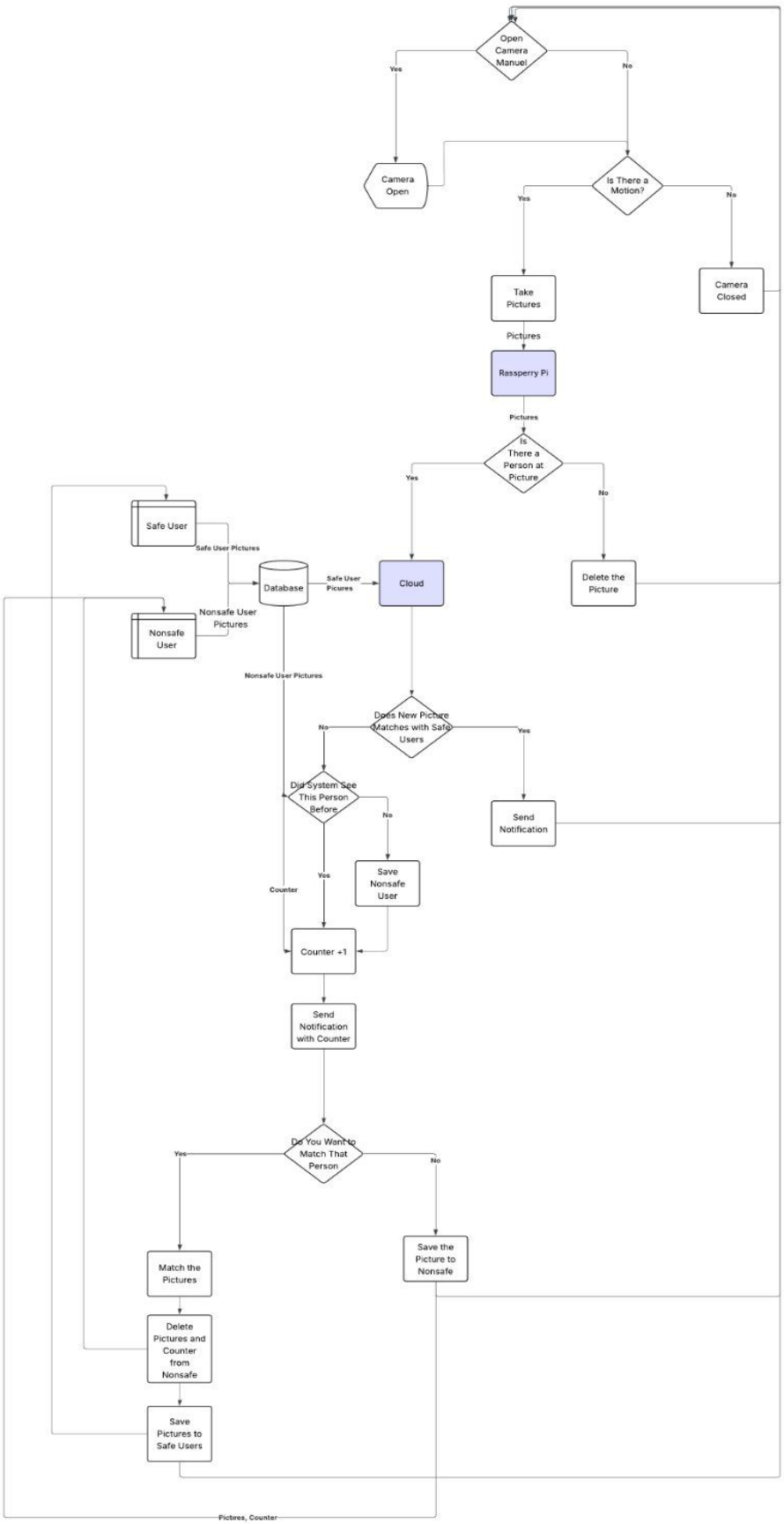

Figure 8 Classifying Authorized/Unauthorized Persons Data-flow Diagram

This figure illustrates the data flow and decision-making process of the surveillance and intrusion analysis module in the smart home system. The workflow begins with either a manual camera request from the user or motion detection by sensors. Captured images are initially processed on the edge device (Raspberry Pi) to detect human presence. If a person is identified, the images are forwarded to the cloud for face recognition by comparing them with registered safe user profiles stored in the database. When no match is found, the system performs a frequency analysis to determine whether the individual has been previously detected, increments a visit counter if necessary, and notifies the user accordingly. Based on user feedback, the detected individual may later be classified as a safe user, allowing the system to update stored profiles and improve future recognition accuracy.

4. Database Design

The database is designed to support user authentication, device management, sensor data storage, and event logging for the IoT-based smart home system. A centralized cloud database is used to ensure data persistence, scalability, and reliable access from both the mobile application and the web dashboard.

4.1 User Authentication Data

User-related data is stored to support secure application access. Basic user information and encrypted credentials are maintained in the database. Upon successful login, a session record is created to manage authenticated access. This structure ensures that only authorized users can interact with the system and access device data.

4.2 Device and Sensor Data

Each registered user may own one or more smart home devices. Devices are associated with multiple sensors, such as temperature, humidity, gas, motion, and load sensors. Sensor readings are stored as time-stamped records to enable real-time monitoring as well as historical data analysis through the client applications.

4.3 Event and Alert Management

System events, including emergency situations (e.g., fire detection), intrusion detection, and low resource levels, are recorded as alerts in the database. Alerts are linked to both the originating device and the corresponding user. This allows users to review recent alerts, acknowledge notifications, and track past incidents through the dashboard interfaces.

4.4 Surveillance Data

Camera-based security events are logged to support the surveillance and intrusion analysis features. Captured images and detection results are associated with camera events. Authorized user face profiles are stored separately and referenced during cloud-based face recognition processes to distinguish known users from unknown individuals.

4.5 Design Considerations

The database design follows a relational structure with clearly defined relationships between users, devices, sensors, and events. This approach ensures data integrity, simplifies access control after authentication, and supports future system extensions without requiring major architectural changes.

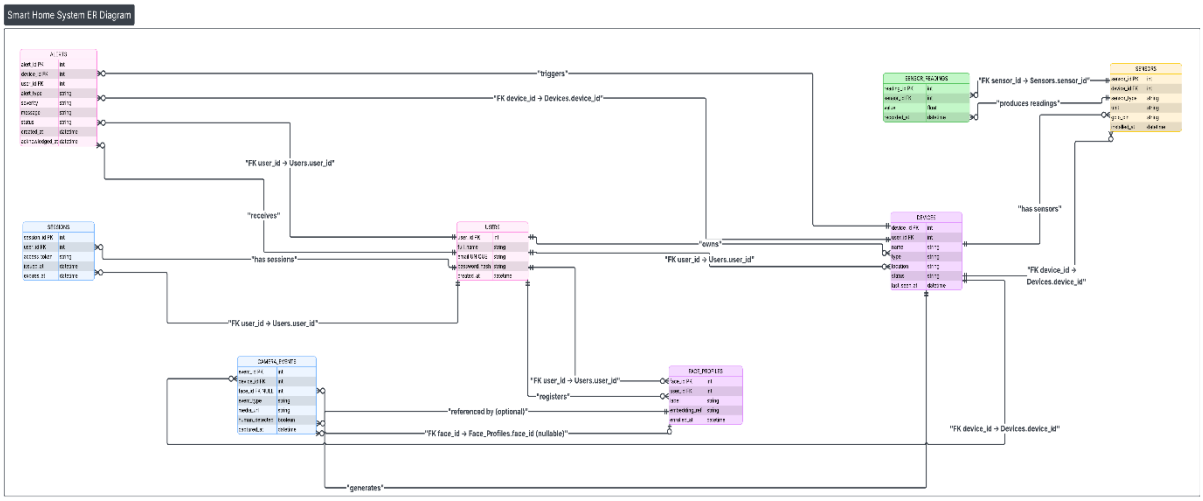

Figure 9 Entity-Relationship (ER) Diagram of the Smart Home System Database

This figure illustrates the Entity-Relationship (ER) diagram of the database designed for the IoT-based smart home system. The diagram presents the core entities, including users, devices, sensors, camera events, and alerts, along with their relationships and cardinalities. Primary and foreign keys are used to ensure data integrity, while optional relationships support scenarios such as unidentified individuals in camera-based surveillance. The database structure supports user authentication, real-time monitoring, event logging, and alert management in a centralized cloud environment.

5. Conclusion

This document presented the architectural design and system components of an IoT-based smart home system that integrates environmental monitoring, automated resource management, and AI-supported security features. By adopting an edge-first approach, the system ensures reliable local operation, reduced latency, and improved privacy while leveraging cloud services for data persistence, advanced processing, and remote accessibility.

The proposed layered architecture provides clear separation of concerns between user interfaces, security, business logic, and data management. Core functionalities such as climate monitoring, emergency response, resource tracking, and surveillance are designed as modular services, enabling scalability and ease of future extension. The surveillance and intrusion analysis workflow combines edge-based processing with cloud-based face recognition to enhance system responsiveness and security awareness.

The database design and data flow structures support secure user authentication, real-time monitoring, event logging, and alert management in a centralized manner. Overall, the proposed design offers a robust and flexible foundation for a smart home system and can be extended in future work to include additional sensors, automation rules, and advanced analytics.